# Лабораторная работа №5
# Тема: «Применение каскадных таблиц стилей CSS»

## ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ:

- формирование умений создания HTML-документов с применением каскадных таблиц стилей.

## ХОД РАБОТЫ:

**Основное понятие**

CSS (Cascading Style Sheets), или каскадные таблицы стилей**,** используются для описания внешнего вида документа, написанного языком разметки. Каскадные таблицы стилей описывают правила форматирования элементов с помощью свойств и допустимых значений этих свойств. Для каждого элемента можно использовать ограниченный набор свойств, остальные свойства не будут оказывать на него никакого влияния.

Каскадность заключается в том, что стили могут переопределяться. Приведенный ниже список способов внедрения стилей соответствует порядку переопределения. Нижерасположенный способ может переопределять вышерасположенный.

**Синтаксис**

Объявление стиля состоит из двух частей: элемента веб-страницы — **селектора**, и команды форматирования — **блока объявления**. Селектор сообщает браузеру, какой именно элемент форматировать, а в блоке объявления (код в фигурных скобках) перечисляются форматирующие команды — свойства и их значения.

**Способы подключения стилей.**

1. Связанные таблицы стилей (Linked Style Sheets). Файл с расширением css подключается к html документу. Это происходит с помощью тега link и атрибута href:

```
<link rel="stylesheet" type="text/css" href="css/style.css">
```

2. Глобальные таблицы стилей (Global Style Sheets). Стили прописываются внутри тега head в теге style.

```
<head>
    <STYLE type="text/css">

        h1 { color:red; font-style:italic; font-size:32px} /*переопределение
стандартного тега.*/
        .blue { color:blue} /*определение нового класса*/
    </STYLE>
</head>
```

3. Внутренние таблицы стилей (Inline Style Sheets). Внутри тегов с помощью атрибутов style в двойных ковычках указываются свойства со значениями.

```
<font style="color:blue; font-size:12pt; font-family:Arial"> Пример </font>
```

**Селекторы**

1. Универсальный селектор *. Применится для всех элементов

```
* {
    margin: 0;
}
```

2. Селектор элементов по названию тега

```
h1 {
    font-family: Lobster, cursive;
}
```

3. Селектор класса**.** Селекторы класса позволяют задавать стили для одного и более элементов с одинаковым именем класса, размещенных в разных местах страницы или на разных страницах сайта. Добавляются с помощью атрибута класс. Обращение через точку.

HTML:

```
<h1 class="headline"></h1>
```

CSS:

```
.headline {
    text-transform: uppercase;
    color: lightblue;
}
```

4. Селектор идентификатора. Селектор идентификатора позволяет форматировать один конкретный элемент. Идентификатор id должен быть уникальным и на одной странице может встречаться только один раз. Добавляются с помощью атрибута id. Обращение происходит через решетку.

HTML:

```
<div id="sidebar"></div>
```

CSS:

```
#sidebar {
    width: 300px;
    float: left;
}
```

5. Селектор псевдокласса. Псевдоклассы — это классы, фактически не прикрепленные к HTML-тегам. Они позволяют применить CSS-правила к элементам при совершении события или подчиняющимся определенному правилу. Часть псевдоклассов:

:link — не посещенная ссылка;
:visited — посещенная ссылка;
:hover — любой элемент, по которому проводят курсором мыши;
:focus — интерактивный элемент, к которому перешли с помощью клавиатуры или активировали посредством мыши;
:active — элемент, который был активизирован пользователем;
:valid — поля формы, содержимое которых прошло проверку в браузере на соответствие указанному типу данных;
:invalid — поля формы, содержимое которых не соответствует указанному типу данных;
:enabled — все активные поля форм;
:disabled — заблокированные поля форм, т.е., находящиеся в неактивном состоянии;
:in-range — поля формы, значения которых находятся в заданном диапазоне;
:out-of-range — поля формы, значения которых не входят в установленный диапазон;
:lang() — элементы с текстом на указанном языке;
:not(селектор) — элементы, которые не содержат указанный селектор — класс, идентификатор, название или тип поля формы — :not([type="submit"]);
:target — элемент с символом #, на который ссылаются в документе;
:checked — выделенные (выбранные пользователем) элементы формы.

## Пример

```
a:link {
    color: #497DDD;
    border-bottom: 1px dashed;
}

a:visited {
    color: #EF7D55;
}

a:hover {
    color: #154088;
    border-bottom: .07em solid;
}

a:active {
    color: #497DDD;
    border-bottom: 1px dashed;
}
```

**Группировка селекторов.** Один и тот же стиль можно одновременно применить к нескольким элементам. Для этого необходимо в левой части объявления перечислить через запятую нужные селекторы:

```
h1, h2, p, span {
    color: tomato;
    background: white;
}
```

**Единицы задания размеров.** В свойствах, которым требуется указание размеров, можно использовать несколько способов для их задания:

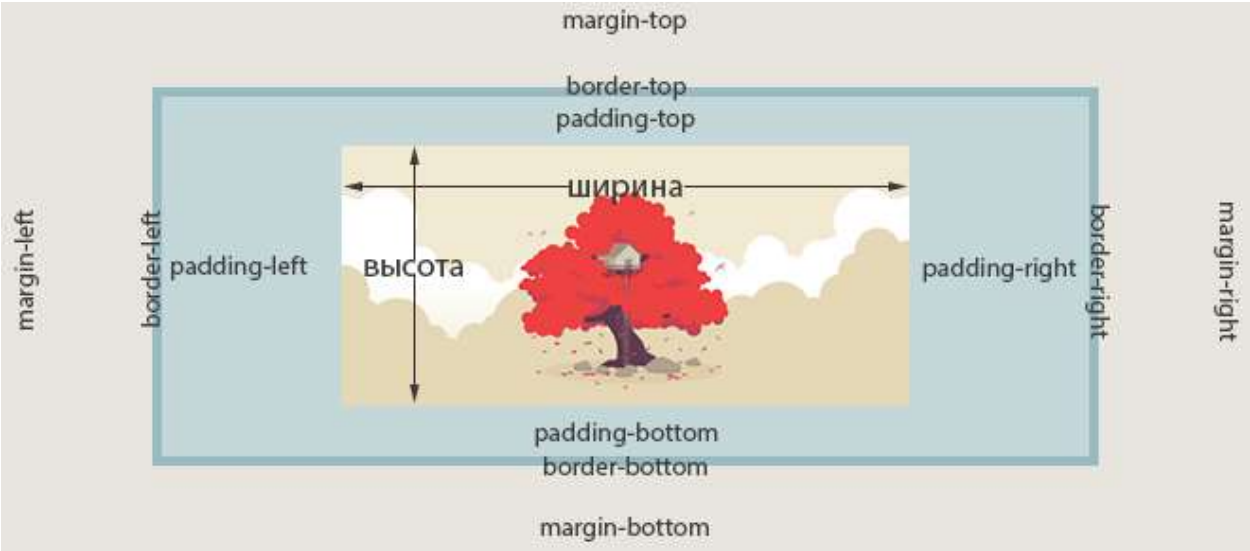
- относительный размер в процентах (%)
- относительный размер при помощи словесного описания (larger, smaller, xx-small, x-small, small, medium, large, x-large, xx-large)
- абсолютный размер в типографских единицах - размер может задаваться в пунктах (pt), пиках (pc), средней шириной буквы "m" (em), средней шириной буквы "x" (ex)
- абсолютный размер в стандартных единицах длины - размер может задаваться в сантиметрах (cm), миллиметрах (mm), дюймах (in)
- абсолютный в пикселях (px)

**Задание цвета в CSS.** Цвет для тех свойств, где это нужно, может быть определен одним из трех способов:

- при помощи названия цвета: yellow, red, green, grey,..
- шестнадцатеричным заданием цвета в формате #RRGGBB: #ff0000, #883490, #ffffff,..
- десятичным заданием составляющих цвета в формате rgb(red, green, blue): rgb(255,0,0), rgb(100,23,78),..

**Блочная модель**

В блочной модели элемент рассматривается как прямоугольный контейнер, имеющий область содержимого и необязательные рамки и отступы (внутренние и внешние). Свойство display определяет тип контейнера элемента. Для каждого элемента существует значение браузера по умолчанию.
Самым популярным блочным элементом является div.




**Область содержимого** — это содержимое элемента, например, текст или изображение.

**Внутренний отступ** задаётся свойством padding. Внутренний отступ — это расстояние между основным содержимым и его границей (рамкой). Если для элемента задать фон, то он распространится также и на поля элемента. Внутренний отступ не может принимать отрицательных значений, в отличие от внешнего отступа.

**Внешний отступ** задаётся свойством margin. Он добавляет расстояние снаружи элемента от внешней границы рамки до соседних элементов, тем самым разделяя элементы на странице. Внешние отступы всегда остаются прозрачными и через них виден фон родительского элемента.

Значения padding и margin задаются в следующем порядке: верхнее, правое, нижнее и левое.

**Граница**, или рамка элемента, задаётся с помощью свойства border. Если цвет рамки не задан, она принимает цвет основного содержимого элемента, например, текста. Если рамка имеет разрывы, то сквозь них будет проступать фон элемента.

Внешние, внутренние отступы и рамка элемента не являются обязательными, по умолчанию их значение равно нулю. Тем не менее, некоторые браузеры добавляют этим свойствам положительные значения по умолчанию на основе своих таблиц стилей. Очистить стили браузеров для всех элементов можно при помощи универсального селектора

Блочные элементы могут размещаться непосредственно внутри элемента <body>. Они создают разрыв строки перед элементом и после него,

образуя прямоугольную область, по ширине занимающую всю ширину веб-страницы или блока-родителя.

Свойства width и height устанавливают ширину и высоту области содержимого элемента. Фактическая ширина элемента складывается из ширины полей (внутренних отступов) padding, границ border и внешних отступов margin.

Свойство position позволяет точно задать новое местоположение блока относительно того места, где он находился бы в нормальном потоке документа. По умолчанию все элементы располагаются последовательно один за другим в том порядке, в котором они определены в структуре HTML-документа. Свойство не наследуется.

**Position**

| | |
|---|---|
| static | Значение по умолчанию, означает отсутствие позиционирования. Элементы отображаются последовательно один за другим в том порядке, в котором они определены в HTML-документе. Используется для очистки любого другого значения позиционирования. |
| relative | Относительно позиционированный элемент сдвигается со своего обычного места в разных направлениях относительно границ родительского контейнера, а пространство, которое он занимал, не исчезает. При этом такой элемент может перекрывать другое содержимое на странице. Относительное позиционирование позволяет задавать z-index для элемента, а также абсолютно позиционировать дочерние элементы внутри блока. |
| absolute | Абсолютно позиционированный элемент полностью удаляется из потока документа и позиционируется относительно границ его блока-контейнера (другого элемента или окна браузера). Блок-контейнер для абсолютно позиционированного элемента — ближайший элемент-предок, значение свойства position которого не равно static.<br>Местоположение краёв элемента определяется с помощью свойств смещения. Пространство, которое занимал такой элемент, схлопывается, как будто элемента не существовало на странице. Абсолютно позиционированный элемент может перекрывать другие элементы или быть перекрытым ими (за счёт свойства z-index). Любой абсолютно позиционированный элемент генерирует блок, то есть принимает значение display: block;. |
| fixed | Фиксирует элемент в указанном месте страницы. Блоком-контейнером фиксированного элемента является окно просмотра, то есть элемент всегда фиксируется относительно окна браузера и не меняет своего положения во время прокрутки страницы. Сам элемент при этом полностью удаляется из основного потока документа и создаётся в новом |

| | |
|---|---|
| | потоке документа. |
| initial | Устанавливает значение свойства в значение по умолчанию. |
| inherit | Наследует значение свойства от родительского элемента. |

Свойство float позволяет перемещать любой элемент, выравнивая его по левому или правому краю веб-страницы или содержащего его элемента-контейнера.

При использовании свойства float для блочных элементов обязательно задавать ширину. Тем самым браузер создаст место для другого содержимого. Но если совокупная ширина всех столбцов окажется больше доступного места, то последний элемент спустится вниз.

| **float** | |
|---|---|
| none | Значение по умолчанию. Также отменяет любое перемещение для элемента из группы элементов, для которых уже установлено обтекание. |
| left | Элемент изымается из нормального потока элементов и позиционируется по левому краю блока-контейнера. |
| right | Элемент позиционируется по правому краю блока-контейнера. |
| initial | Устанавливает значение свойства в значение по умолчанию. |
| inherit | Наследует значение свойства от родительского элемента. |

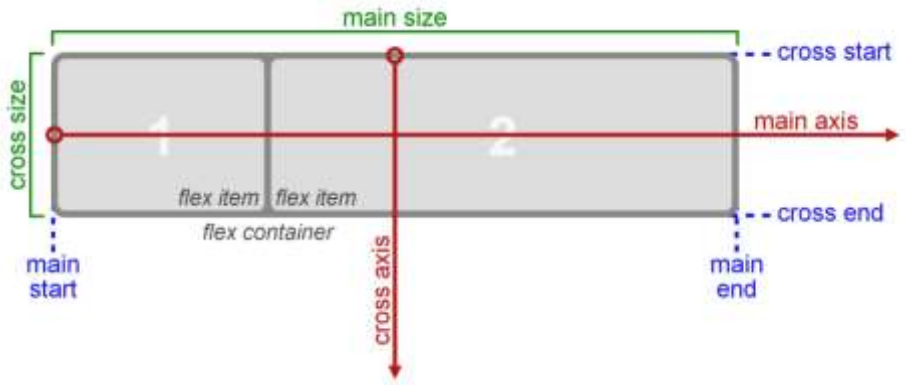
**Флекс бокс**

Флекс-бокс -- механизм для построения сложной раскладки элементов

все блоки могут растягиваться и сжиматься в зависимости от заданых параметров.

выравниваться по вертикали, горизонтали.

В основе flexbox лежит идея оси. Flexbox является инструментом двумерной компоновки и использует для работы две оси — горизонтальную (главную ось) и поперечную.




## Для инициализации контейнера нужно элементу через css указать

```
display:flex
```

## Пример

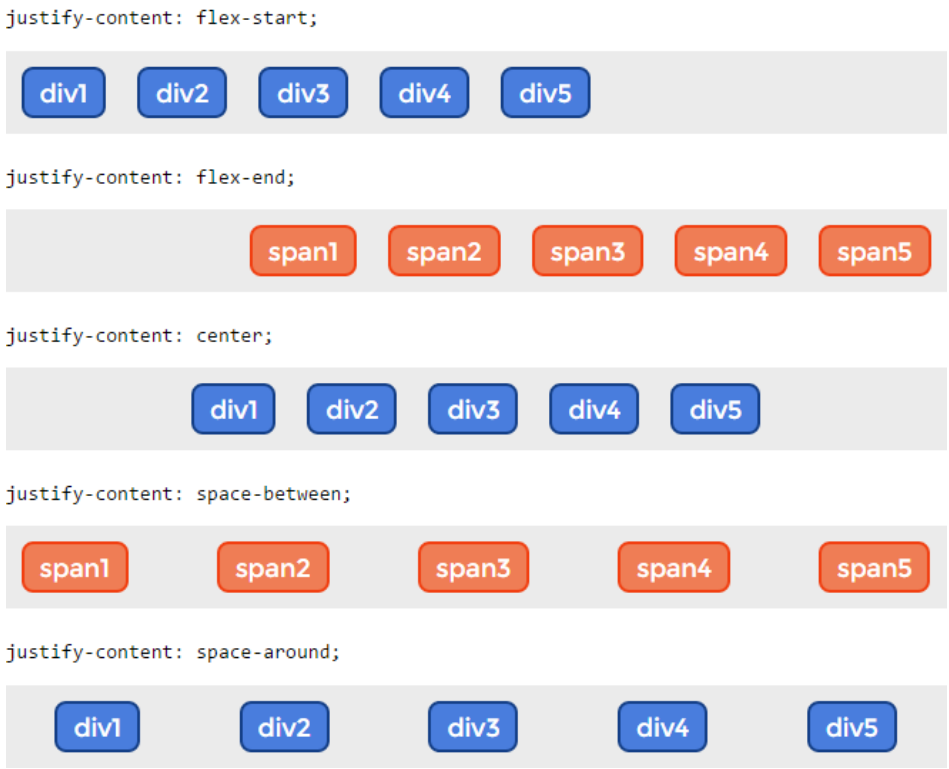
HTML:

```
<div class="my-flex-container">
    <div class="my-flex-
block">item1</div>
    <div class="my-flex-
block">item2</div>
    <div class="my-flex-
block">item3</div>
</div>
```

CSS:

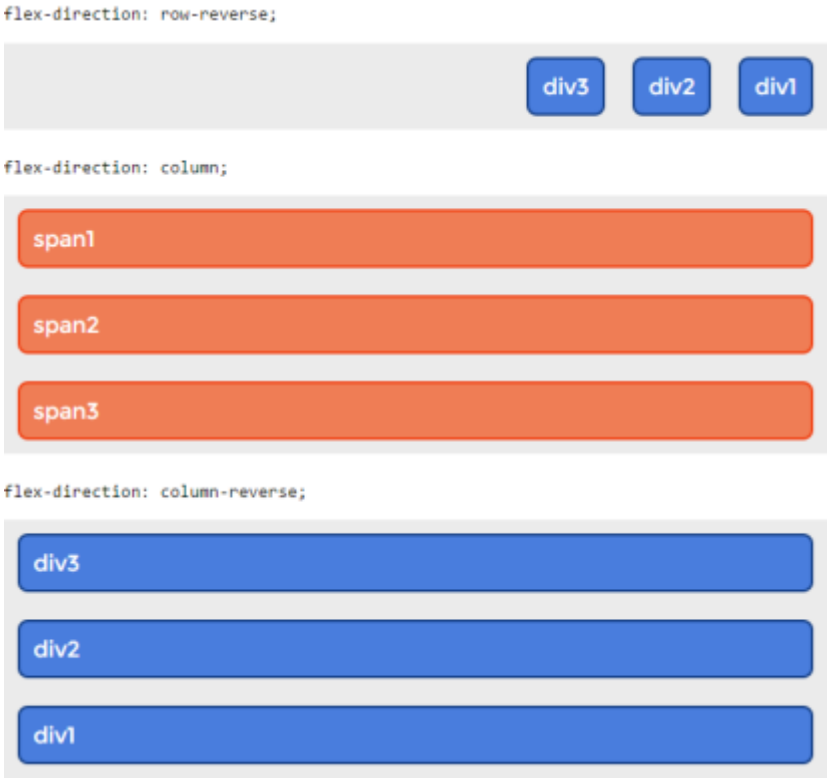
```
.my-flex-container{
    display: flex;
}
```

## Свойство Justify-content



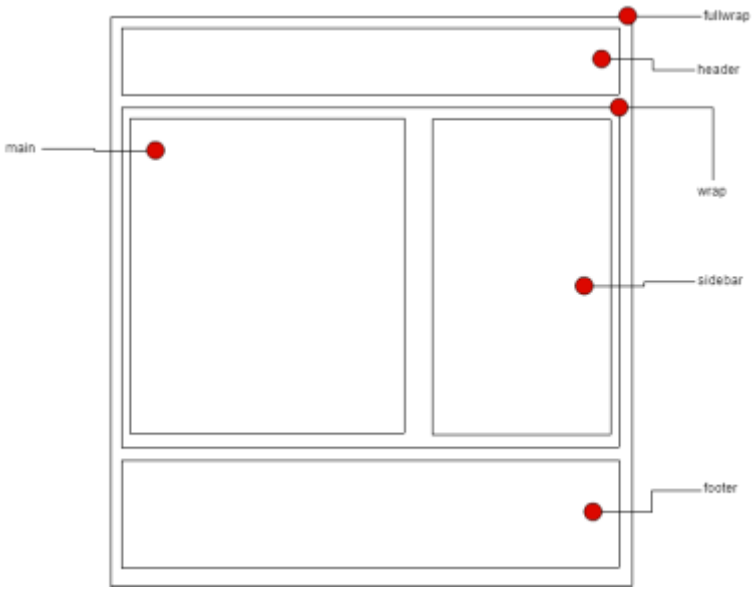
## Свойство align-items:

## Свойство flex-direction:



## ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ:

1. Создайте страницу из элементов как показано на картинке с помощью блочной модели:




2. Задайте фон, шрифты, цвет текста (переопределив, тег <body>).
3. Установите разные цвета блокам и сделайте для них рамку.
4. В блоке header разместите логотип и название компании по варианту.
5. В блоке footer разместите свои ФИО и дату.
6. В блоке main разместите картинки и текст с описанием компании. Представьте их с разным обтеканием текста.
7. В блоке sidebar разместите 3 ссылки: History, Team, Contact. Примените к ссылкам изменение цвета при наведении.

Вариант определяется по формуле: Вариант = ((номер студента в журнале) mod 10) + 1

Варианты:

1. Apple
2. Google
3. Microsoft
4. Amazon
5. Walmart
6. Facebook
7. Twitter
8. LinkedIn
9. Pinterest
10. Instagram